

INSTITUTO POLITÉCNICO NACIONAL
Escuela Superior de Comercio y Administración
Unidad Santo Tomás

Práctica 5 y 6 en proyecto - Pugnela

Equipo 5 - Pugnela

- **Escobar Hernández Edna 100%**
- **González Torres Fernanda 100%**
 - **López Reyes Fernanda 100%**
 - **Moreno Ruiz Diana Yael 100%**

Unidad de aprendizaje: Laboratorio empresarial.

Grupo: 6GM1

Miércoles 3 de mayo de 2026.

Periodo escolar 2026/1

Práctica 5: Controlador del Carrito

Introducción

En esta práctica se desarrolló el controlador del carrito de compras para el sistema web Pugnela, con la finalidad de permitir que los usuarios agreguen productos desde la página principal hacia un carrito digital. Esta función es importante porque representa una parte básica de una tienda en línea, ya que permite seleccionar productos, indicar cantidades y conservarlos durante la sesión del usuario.

El desarrollo se realizó utilizando PHP, MySQL, XAMPP y la arquitectura MVC, separando la lógica del sistema en modelos, vistas y controladores. De esta manera, el proyecto mantiene una estructura más organizada y fácil de modificar.

Desarrollo

Primero se verificó el funcionamiento del modelo Cart.php, el cual se encarga de realizar las operaciones principales con la tabla carrito en la base de datos. Este modelo permite agregar productos, consultar los productos guardados y calcular el total del carrito.

Código del modelo Cart.php

```
<?php
```

```
require_once 'config/database.php';
```

```
class Cart {
```

```
    private $conn;
```

```
    private $table_name = "carrito";
```

```
    public function __construct() {
```

```
        $database = new Database();
```

```
        $this->conn = $database->getConnection();
```

```
    }
```

```
    public function addToCart($usuario_id, $producto_id, $cantidad = 1) {
```

```
$query = "SELECT * FROM " . $this->table_name . "  
        WHERE usuario_id = :usuario_id  
        AND producto_id = :producto_id";
```

```
$stmt = $this->conn->prepare($query);  
$stmt->bindParam(':usuario_id', $usuario_id);  
$stmt->bindParam(':producto_id', $producto_id);  
$stmt->execute();
```

```
if($stmt->rowCount() > 0) {  
    $query = "UPDATE " . $this->table_name . "  
            SET cantidad = cantidad + :cantidad  
            WHERE usuario_id = :usuario_id  
            AND producto_id = :producto_id";  
} else {  
    $query = "INSERT INTO " . $this->table_name . "  
            (usuario_id, producto_id, cantidad)  
            VALUES  
            (:usuario_id, :producto_id, :cantidad)";  
}
```

```
$stmt = $this->conn->prepare($query);  
$stmt->bindParam(':usuario_id', $usuario_id);  
$stmt->bindParam(':producto_id', $producto_id);  
$stmt->bindParam(':cantidad', $cantidad);
```

```
return $stmt->execute();
```

```
}
```

```
public function getCartItems($usuario_id) {
```

```
    $query = "SELECT c.*, p.nombre, p.precio, p.imagen
```

```
        FROM " . $this->table_name . " c
```

```
        JOIN productos p
```

```
        ON c.producto_id = p.id
```

```
        WHERE c.usuario_id = :usuario_id";
```

```
    $stmt = $this->conn->prepare($query);
```

```
    $stmt->bindParam(':usuario_id', $usuario_id);
```

```
    $stmt->execute();
```

```
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
}
```

```
public function getCartTotal($usuario_id) {
```

```
    $items = $this->getCartItems($usuario_id);
```

```
    $total = 0;
```

```
    foreach($items as $item) {
```

```
        $total += $item['precio'] * $item['cantidad'];
```

```
    }
```

```
    return $total;
```

```
}
```

```

public function clearCart($usuario_id) {

    $query = "DELETE FROM " . $this->table_name . "
        WHERE usuario_id = :usuario_id";

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':usuario_id', $usuario_id);

    return $stmt->execute();
}
}

?>

```

Después se creó el archivo CartController.php, el cual funciona como intermediario entre el modelo del carrito y las vistas. Este controlador recibe las acciones del usuario, como agregar un producto o visualizar el carrito, y llama a los métodos correspondientes del modelo.

Código del controlador CartController.php

```

<?php

require_once 'models/Cart.php';

class CartController {

    private $cartModel;

    public function __construct() {
        $this->cartModel = new Cart();
    }
}

```

```
}
```

```
public function addToCart() {
```

```
    if($_SERVER['REQUEST_METHOD'] == 'POST') {
```

```
        $usuario_id = $_SESSION['user_id'];
```

```
        $producto_id = $_POST['product_id'];
```

```
        $cantidad = $_POST['cantidad'] ?? 1;
```

```
        if($this->cartModel->addToCart($usuario_id, $producto_id, $cantidad)) {
```

```
            $_SESSION['success'] = "✅ Producto agregado al carrito";
```

```
        } else {
```

```
            $_SESSION['error'] = "❌ Error al agregar producto";
```

```
        }
```

```
        header('Location: index.php?action=dashboard');
```

```
        exit();
```

```
    }
```

```
}
```

```
public function viewCart() {
```

```
    $usuario_id = $_SESSION['user_id'];
```

```
    $cartItems = $this->cartModel->getCartItems($usuario_id);
```

```
    $total = $this->cartModel->getCartTotal($usuario_id);
```

```
    include 'views/cart/index.php';
```

```
}  
}
```

```
?>
```

También se modificó el archivo index.php para integrar las rutas relacionadas con el carrito, como cart y add-to-cart. Esto permitió que el sistema reconociera las nuevas acciones y dirigiera al usuario a la función correspondiente.

Código de index.php

```
<?php
```

```
session_start();
```

```
require_once __DIR__ . '/controllers/AuthController.php';
```

```
require_once __DIR__ . '/controllers/ProductController.php';
```

```
require_once __DIR__ . '/controllers/CartController.php';
```

```
$action = isset($_GET['action']) ? $_GET['action'] : 'login';
```

```
$authController = new AuthController();
```

```
$productController = new ProductController();
```

```
$cartController = new CartController();
```

```
switch($action) {
```

```
    case 'login':
```

```
        if($_SERVER['REQUEST_METHOD'] == 'POST') {
```

```
            $authController->login();
```

```
        } else {
```

```
    $authController->showLogin();  
}
```

```
break;
```

```
case 'dashboard':
```

```
    $authController->checkAuth();  
    $productController->dashboard();
```

```
break;
```

```
case 'cart':
```

```
    $authController->checkAuth();  
    $cartController->viewCart();
```

```
break;
```

```
case 'add-to-cart':
```

```
    $authController->checkAuth();  
    $cartController->addToCart();
```

```
break;
```

```
case 'logout':
```



```
$authController->logout();
```

```
break;
```

default:

```
$authController->showLogin();
```

```
break;
```

```
}
```

```
?>
```

Posteriormente se actualizó la página de inicio o dashboard para incluir el botón de “Agregar al carrito” en cada producto. Además, se agregó un campo de cantidad para que el usuario pueda seleccionar cuántas piezas desea agregar.

Código agregado en el dashboard

```
<form action="index.php?action=add-to-cart" method="POST">
```

```
  <input type="hidden" name="product_id" value="<?php echo $product['id']; ?>">
```

```
  <input
```

```
    class="cantidad"
```

```
    type="number"
```

```
    name="cantidad"
```

```
    value="1"
```

```
    min="1"
```

```
    max="<?php echo $product['stock']; ?>">
```

```
  <button type="submit" <?php echo $product['stock'] <= 0 ? 'disabled' : ''; ?>>
```

```
    <img alt="Shopping cart icon" data-bbox="180 878 205 897"/> Agregar al carrito
```

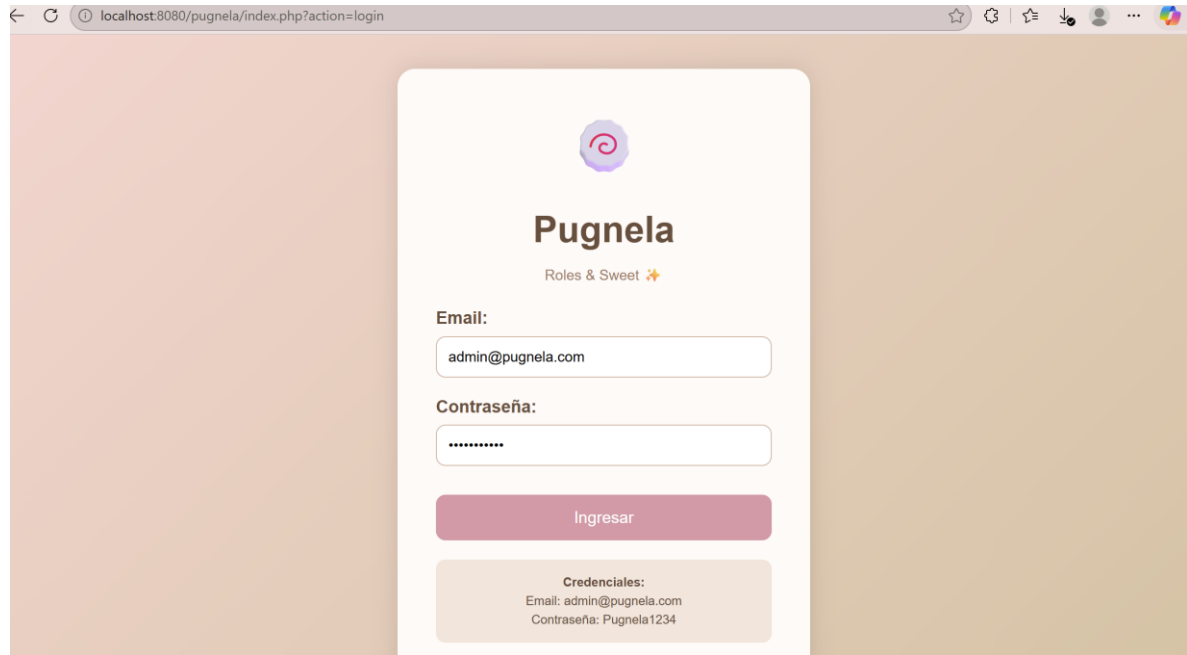
</button>

</form>

Finalmente se creó la vista del carrito en views/cart/index.php, donde se muestran los productos agregados, su precio, cantidad, subtotal y total general.

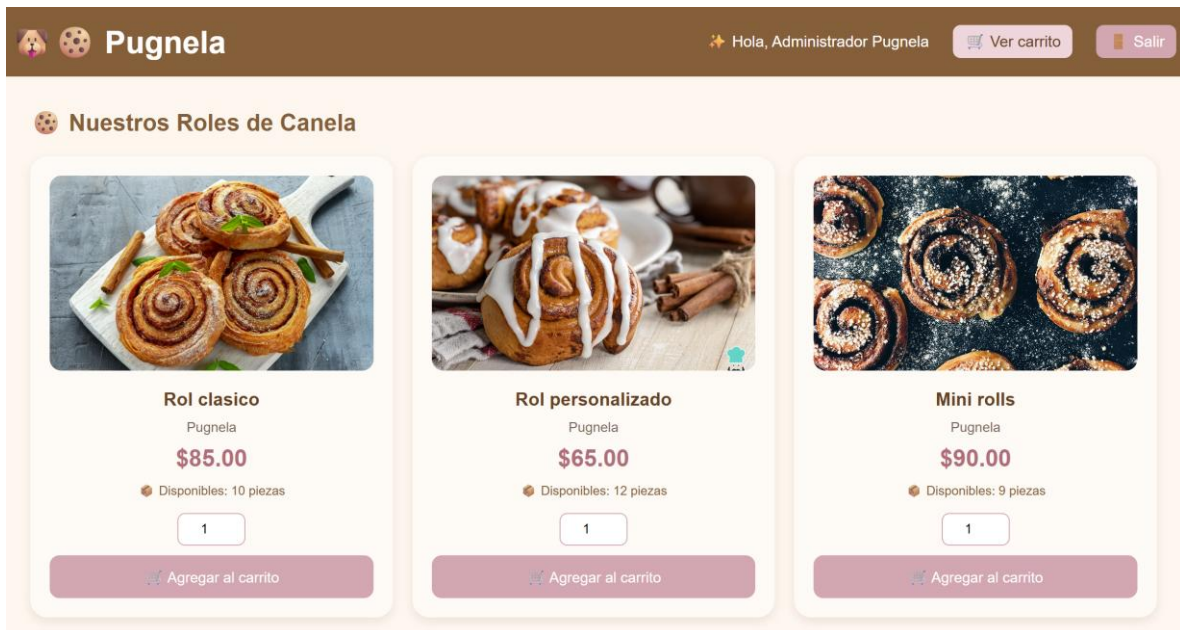
Evidencias de funcionamiento

Imagen 1. Login



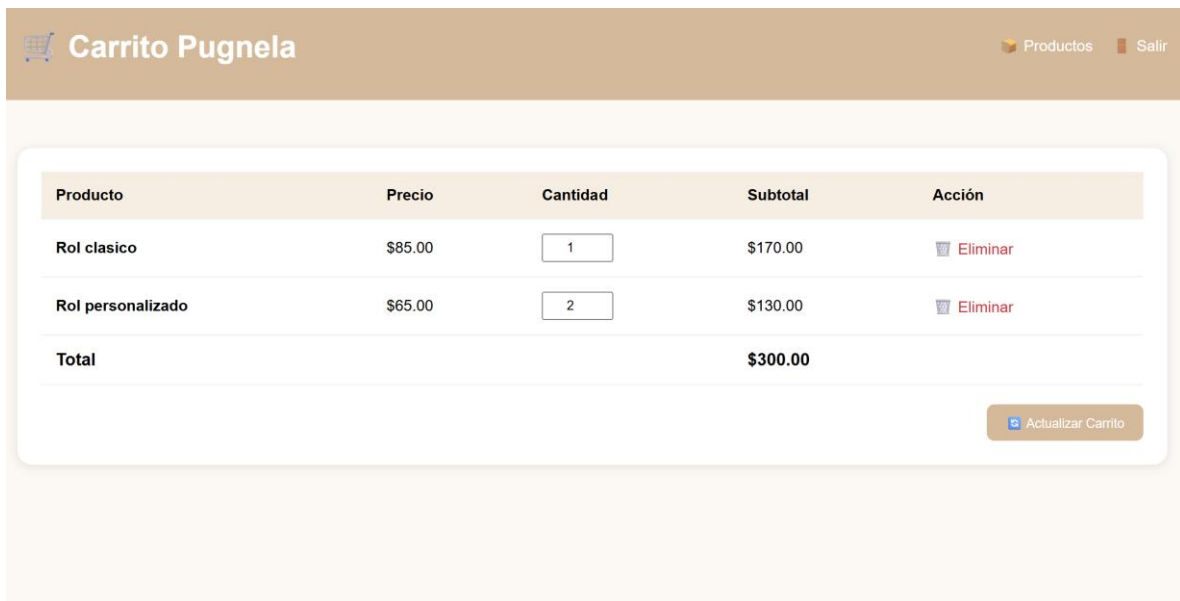
En esta captura se muestra la pantalla inicial de acceso al sistema Pugnela. Esta sección permite que el usuario ingrese con sus datos para acceder al dashboard y utilizar las funciones del sistema.

Imagen 2. Página de inicio



En esta imagen se observa el dashboard principal de Pugnela, donde se muestran los productos disponibles. Cada producto cuenta con su precio, stock y el botón para agregarlo al carrito.

Imagen 3. Carrito Pugnela



Esta captura muestra la vista del carrito de compras después de agregar productos. Se observan los productos seleccionados, el precio de cada uno, la cantidad, el subtotal y el total general de la compra.

Conclusión

Con esta práctica se logró implementar el controlador del carrito de compras dentro del sistema Pugnela. Esta función permitió agregar productos desde el dashboard, guardar la información en la base de datos y mostrar los productos seleccionados en una vista de carrito. Además, se reforzó el uso de la arquitectura MVC, el manejo de sesiones y la conexión entre PHP y MySQL.

Práctica 6: Funcionalidades Completas del Carrito

Introducción

En esta práctica se complementó el carrito de compras desarrollado previamente, agregando funciones más completas para mejorar la experiencia del usuario. El objetivo principal fue permitir que los productos dentro del carrito pudieran actualizarse o eliminarse, además de recalcular automáticamente el total de la compra.

Estas funcionalidades son necesarias en una tienda en línea, ya que el usuario debe tener control sobre los productos que desea comprar antes de finalizar su pedido.

Desarrollo

Para iniciar, se agregaron nuevos métodos al modelo Cart.php, los cuales permiten actualizar la cantidad de productos y eliminar productos del carrito. Estos métodos trabajan directamente con la tabla carrito de la base de datos.

Métodos agregados al modelo Cart.php

```
public function updateQuantity($cart_id, $usuario_id, $cantidad) {
```

```
    $query = "UPDATE " . $this->table_name . "
```

```
        SET cantidad = :cantidad
```

```
        WHERE id = :cart_id
```

```
        AND usuario_id = :usuario_id";
```

```
    $stmt = $this->conn->prepare($query);
```

```
    $stmt->bindParam(':cantidad', $cantidad);
```

```
    $stmt->bindParam(':cart_id', $cart_id);
```

```
    $stmt->bindParam(':usuario_id', $usuario_id);
```

```
    return $stmt->execute();
```

```
}
```

```
public function removeFromCart($cart_id, $usuario_id) {
```

```

$query = "DELETE FROM " . $this->table_name . "
        WHERE id = :cart_id
        AND usuario_id = :usuario_id";

$stmt = $this->conn->prepare($query);
$stmt->bindParam(':cart_id', $cart_id);
$stmt->bindParam(':usuario_id', $usuario_id);

return $stmt->execute();
}

```

Después se actualizaron los métodos del controlador CartController.php, agregando las funciones necesarias para eliminar productos y actualizar las cantidades desde la vista del carrito.

Métodos agregados al controlador CartController.php

```

public function removeFromCart() {

    if(isset($_GET['id'])) {

        $usuario_id = $_SESSION['user_id'];
        $cart_id = $_GET['id'];

        $this->cartModel->removeFromCart($cart_id, $usuario_id);

        $_SESSION['success'] = "🗑️ Producto eliminado";

        header('Location: index.php?action=cart');
        exit();
    }
}

```

```

public function updateCart() {

    if($_SERVER['REQUEST_METHOD'] == 'POST') {

        $usuario_id = $_SESSION['user_id'];

        foreach($_POST['quantities'] as $cart_id => $cantidad) {

            if($cantidad > 0) {
                $this->cartModel->updateQuantity($cart_id, $usuario_id, $cantidad);
            }
        }

        $_SESSION['success'] = "🔄 Carrito actualizado";

        header('Location: index.php?action=cart');
        exit();
    }
}

```

También se agregaron nuevas rutas en el archivo index.php, para que el sistema pudiera reconocer las acciones de actualizar y eliminar productos del carrito.

Rutas agregadas en index.php

```

case 'update-cart':

    $authController->checkAuth();
    $cartController->updateCart();

    break;

```

```
case 'remove-from-cart':
```

```
    $authController->checkAuth();
```

```
    $cartController->removeFromCart();
```

```
    break;
```

Por último, se mejoró la vista del carrito para incluir botones de acción. El usuario puede cambiar la cantidad de un producto, actualizar el carrito o eliminar un producto específico.

Código principal de la vista del carrito

```
<form action="index.php?action=update-cart" method="POST">
```

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>Producto</th>
```

```
<th>Precio</th>
```

```
<th>Cantidad</th>
```

```
<th>Subtotal</th>
```

```
<th>Acción</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<?php foreach($cartItems as $item): ?>
```


<tr>

<td>

<?php echo \$item['nombre']; ?>

</td>

<td>

\$<?php echo number_format(\$item['precio'],2); ?>

</td>

<td>

<input

type="number"

name="quantities[<?php echo \$item['id']; ?>]"

value="<?php echo \$item['cantidad']; ?>"

min="1">

</td>

<td>

\$<?php echo number_format(\$item['precio'] * \$item['cantidad'],2); ?>

</td>

<td>

<a

class="btn-remove"

href="index.php?action=remove-from-cart&id=<?php echo \$item['id']; ?>"

onclick="return confirm('¿Eliminar producto?')">

 Eliminar

</td>

</tr>

<?php endforeach; ?>

</tbody>

<tfoot>

<tr class="total-row">

<td colspan="3">Total</td>

<td colspan="2">

\$<?php echo number_format(\$total,2); ?>

</td>

</tr>

</tfoot>

</table>

<div class="cart-actions">

<button type="submit" class="btn-update">

 Actualizar Carrito

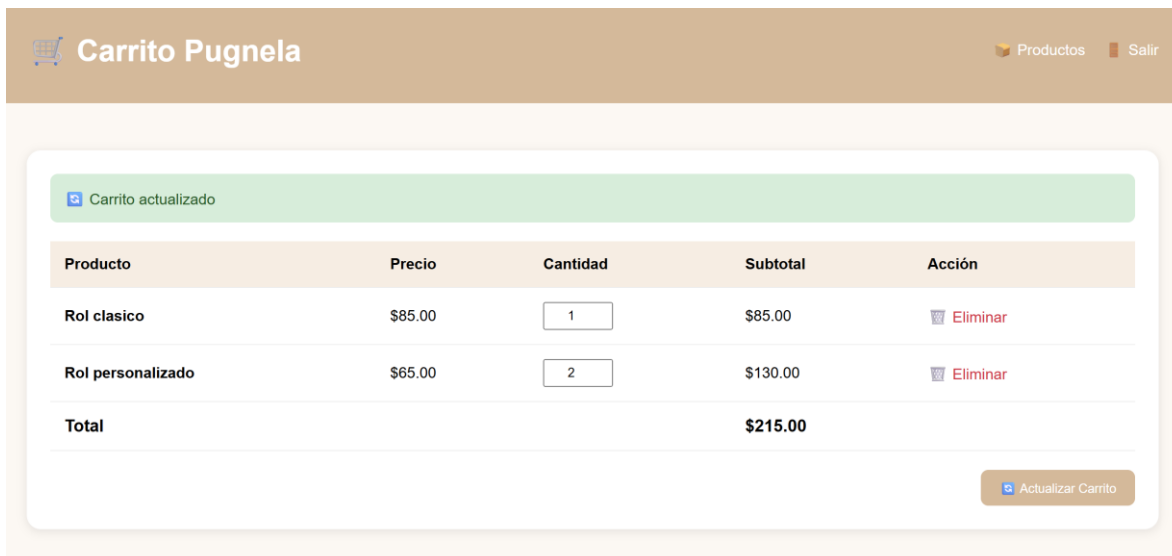
</button>

</div>

</form>

Evidencias de funcionamiento

Imagen 4. Actualizar carrito



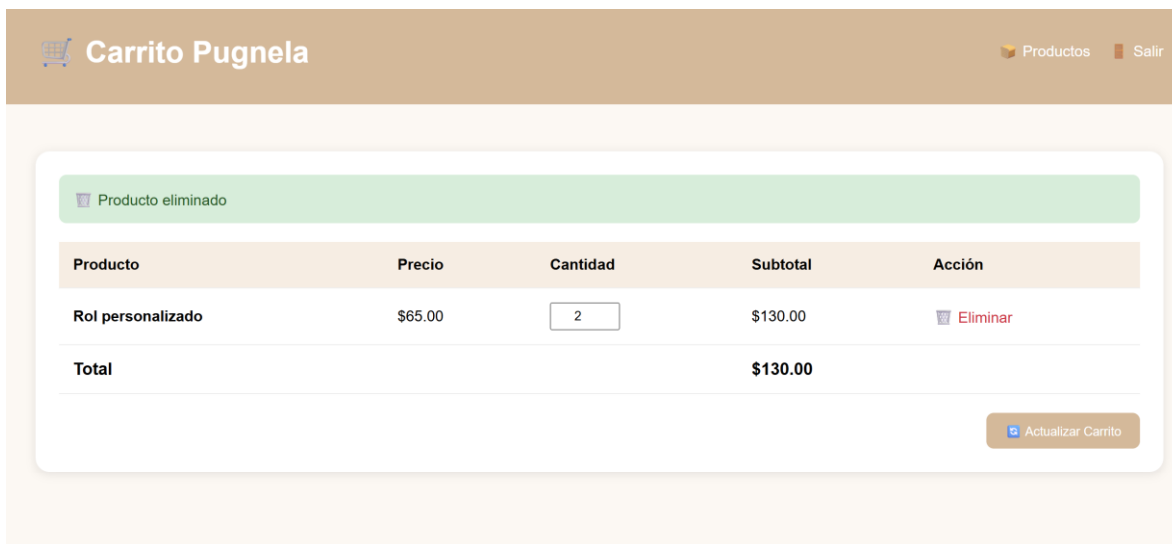
The screenshot shows the 'Carrito Pugnela' interface. At the top, there is a header with a shopping cart icon and the text 'Carrito Pugnela'. To the right of the header are links for 'Productos' and 'Salir'. Below the header, a green notification bar displays 'Carrito actualizado'. Underneath this, a table lists the items in the cart:

Producto	Precio	Cantidad	Subtotal	Acción
Rol clasico	\$85.00	1	\$85.00	Eliminar
Rol personalizado	\$65.00	2	\$130.00	Eliminar
Total			\$215.00	

At the bottom right of the table area, there is a button labeled 'Actualizar Carrito'.

En esta captura se muestra cómo el usuario puede modificar la cantidad de productos agregados al carrito. Al presionar el botón “Actualizar Carrito”, el sistema recalcula los subtotales y el total general.

Imagen 6. Producto eliminado



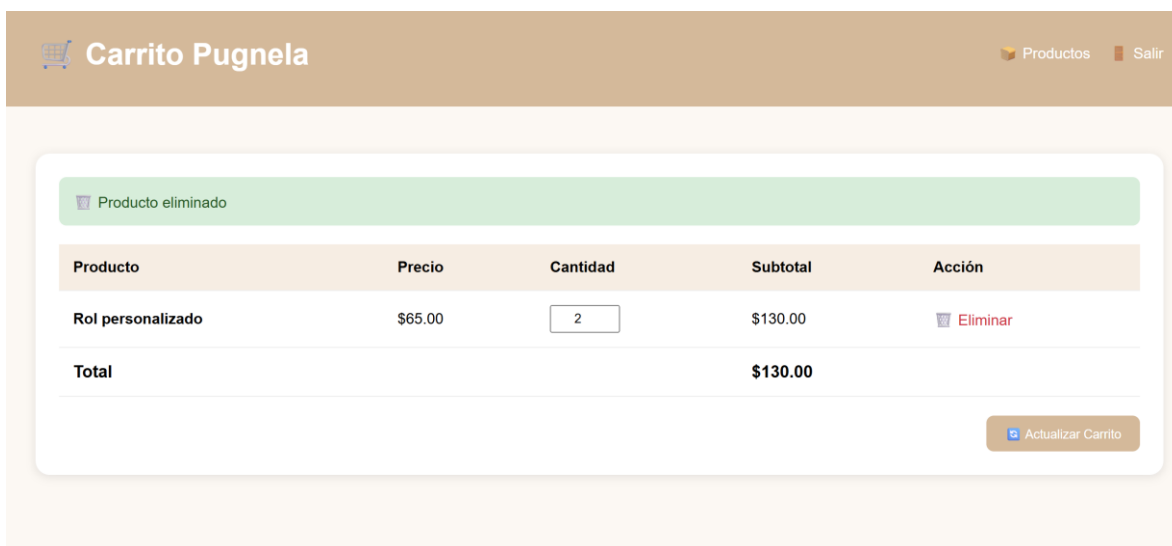
The screenshot shows the 'Carrito Pugnela' interface after a product has been removed. The header and navigation links remain the same. A green notification bar displays 'Producto eliminado'. The table below shows the remaining item:

Producto	Precio	Cantidad	Subtotal	Acción
Rol personalizado	\$65.00	2	\$130.00	Eliminar
Total			\$130.00	

The 'Actualizar Carrito' button is still present at the bottom right.

En esta captura se observa el mensaje de confirmación que indica que el producto fue eliminado correctamente del carrito.

Imagen 7. Carrito actualizado sin el producto



Esta imagen muestra el carrito después de eliminar un producto. Se puede observar que el total se actualiza automáticamente y solo permanece el producto restante.

Conclusión

Con esta práctica se completaron las funcionalidades principales del carrito de compras de Pugnela. Ahora el usuario puede agregar productos, modificar cantidades, eliminar artículos y visualizar el total actualizado de su compra. Esto permitió mejorar la funcionalidad del sistema y acercarlo más a una tienda en línea real.

Además, esta práctica fortaleció el uso de PHP, MySQL, formularios, sesiones y arquitectura MVC, ya que cada acción del carrito se integró correctamente entre modelo, vista, controlador y base de datos.